

Стать IT-шником может каждый



Простые операции с DataFrame



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Создание DataFrame
- Анализ DataFrame
- Индексация в DataFrame



Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Введение

Изменение таблиц

Работа с пропусками

Простые операции

Чтение и запись данных

Интервалы времени



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



Стать IT-шником может каждый



Изменение таблиц



Добавление новых столбцов к таблице

```
d = {'one': pd.Series(range(6), index=list('abdefg')),  
     'two': pd.Series(range(7), index=list('abcdefg')),  
     'three': pd.Series(np.random.normal(size=7), index=list('abcdefg'))}  
df = pd.DataFrame(d)  
df # используем для примера таблицу с прошлого занятия  
  
df['4th'] = df['one'] * df['two']  
df['flag'] = df['two'] > 2  
df['ones'] = 1  
df
```

Удаление имеющихся в таблице столбцов

```
del df['two']  
df['foo'] = 0  
df
```

Изменение элемента

```
df.iat[1, 0] = -1  
# Эквивалентные формы:  
# df['one']['b'] = -1 <-- SettingWithCopyWarning  
# df.at['b', 'one'] = -1  
df
```

Добавление копии столбца one

```
df['one_tr'] = df['one'][:5]  
df
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



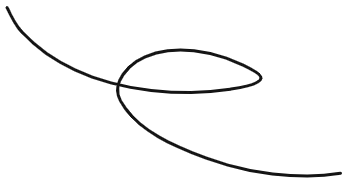
Работа с пропусками





Работа с пропусками —

это важная часть предварительной обработки данных при использовании Pandas DataFrame. Пропущенные значения могут возникнуть по разным причинам, например, из-за ошибок в данных, отсутствия информации или при объединении наборов данных.



Работа с пропусками

**Обнаружение
пропущенных
значений**

**Удаление
пропущенных
значений**

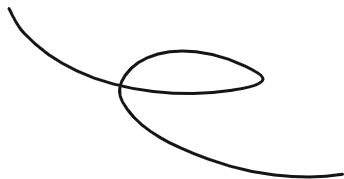
**Заполнение
пропущенных
значений**

**Интерполяция
пропущенных
значений**



isnull() или notnull() —

это методы, которые можно использовать для обнаружения пропущенных значений в DataFrame. Возвращают DataFrame той же структуры, что и исходный, но каждый элемент заменяется на True (если значение пропущено) или False (если значение присутствует).





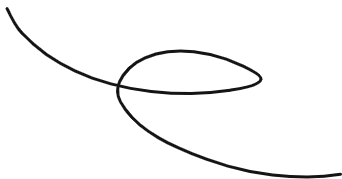
Метод isnull()

```
df.isnull() # Булевская маска пропущенных значений
```




dropna() —

это метод, который можно использовать для удаления строк или столбцов, содержащих пропущенные значения. Позволяет гибко настраивать удаление и задать пороговое количество непропущенных значений для удаления.





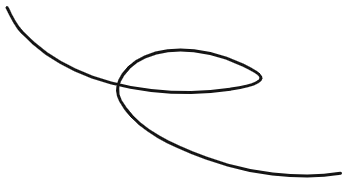
Метод dropna()

```
df.dropna(how='any') # Удаление всех строк с пропусками
```



fillna() —

это метод, который используется для замены пропущенных значений на конкретное значение или на результат выполнения функции. Позволяет заполнять пропуски фиксированным значением, средним, медианным или модальным значением по столбцу или строке и применять более сложные стратегии заполнения.





Метод fillna()

```
df.fillna(value=123) # Замена всех пропусков на значение
```



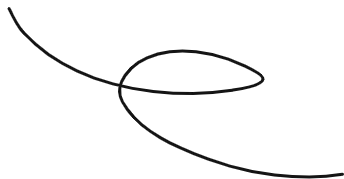
Метод fillna()

```
df.fillna(value=df.mean()) # Замена всех пропусков на среднее по столбцу
```



interpolate() —

это метод, который позволяет заполнить пропущенные значения, используя различные методы интерполяции.



Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Простые операции



Рассмотрим простые операции над DataFrame

```
df1 = pd.DataFrame(np.random.uniform(size=(10, 4)),  
                    columns=['A', 'B', 'C', 'D'])  
df1 # создадим таблицу из массива случайных чисел.
```

Рассмотрим простые операции над DataFrame

```
df2 = pd.DataFrame(np.random.uniform(size=(7, 3)),  
                    columns=['A', 'B', 'C'])  
df2 # и еще одну.
```

Рассмотрим простые операции над DataFrame

`df1 + df2`

Рассмотрим простые операции над DataFrame

```
2 * df1 + 3 # да, так можно!
```


Рассмотрим простые операции над DataFrame

```
np.sin(df1) # и даже так
```

Рассмотрим простые операции над DataFrame

```
np.exp(df1['A']) # можем применить для одного столбца (помним, что это Series)
```

Рассмотрим простые операции над DataFrame

```
df1.cumsum() # старые знакомые – кумулятивные суммы
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



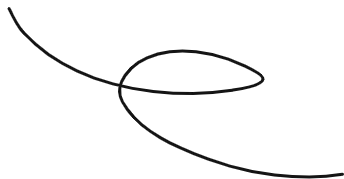
Чтение и запись данных





pd.read_csv—

это функция, с помощью которой производится загрузка текстовых файлов табличного вида.



Функция `pd.read_csv`

Основные аргументы	Пояснение
<code>filepath_or_buffer</code>	Путь к файлу
<code>sep</code>	Разделитель колонок в строке (запятая, табуляция и т.д.)
<code>header</code>	Номер строки или список номеров строк, используемых в качестве имен колонок
<code>names</code>	Список имен, которые будут использованы в качестве имен колонок
<code>index_col</code>	Колонка, используемая в качестве индекса
<code>usecols</code>	Список имен колонок, которые будут загружены
<code>nrows</code>	Сколько строк прочитать
<code>skiprows</code>	Номера строк с начала, которые нужно пропустить

Функция `pd.read_csv`

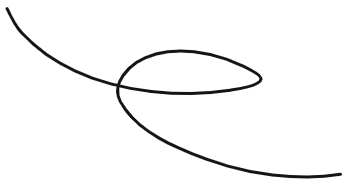
Основные аргументы	Пояснение
<code>skipfooter</code>	Сколько строк в конце пропустить
<code>na_values</code>	Список значений, которые распознавать как пропуски
<code>parse_dates</code>	Распознавать ли даты, можно передать номера строк
<code>date_parser</code>	Парсер дат
<code>dayfirst</code>	День записывается перед месяцем или после
<code>thousands</code>	Разделитель тысяч
<code>decimal</code>	Разделитель целой и дробной частей
<code>comment</code>	Символ начала комментария

Полезно знать



df.to_csv —

это функция, с помощью которой происходит запись таблицы в текстовый файл.



Функция df.to_csv

Основные аргументы	Пояснение
df	DataFrame, который нужно записать
path_or_buf	Путь, куда записать
sep	Разделитель колонок в строке (запятая, табуляция и т.д.)
na_rep	Как записать пропуски
float_format	Формат записи дробных чисел
columns	Какие колонки записать
header	Как назвать колонки при записи
index	Записывать ли индексы в файл
index_label	Имена индексов, которые записать в файл

Пример

```
import datetime as DT

start_date = DT.datetime(2019, 1, 1)
end_date = DT.datetime(2020, 12, 31)

dates = pd.date_range(
    min(start_date, end_date),
    max(start_date, end_date)
).strftime('%Y-%m-%d').tolist()
df = pd.DataFrame({'Time': dates, 'Value': np.random.randint(1, 100, (len(dates), ))})
df.to_csv('./example.csv', sep='\t', index=False)
df.to_csv('./example.tsv', sep='\t', index=False, header=False)
df = pd.read_csv('./example.tsv', sep='\t', parse_dates=[0], names=['Time', 'Value'])
df.head()
df.dtypes # В информации о таблице видим, что дата определилась, т.к. формат колонки
Time обозначен как datetime64[ns].
```

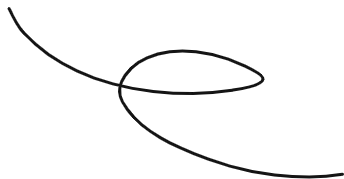
Пример

```
df = pd.read_csv('./example.csv', sep='\t')  
df.info()  
df['Time'] = pd.to_datetime(df['Time'], dayfirst=True) # Тогда можно воспользоваться функцией  
pd.to_datetime
```



pd.read_excel —

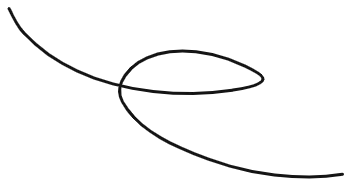
это **функция**, с помощью которой производится загрузка таблиц формата Excel.





df.to_excel—

это **функция**, с помощью которой производится запись таблицы в формат Excel.



Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Интервалы времени

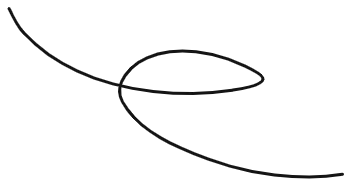


Полезно знать



pd.Timedelta —

это объект, которым задается интервал времени.



Возможные обозначения интервалов



- 'Y', 'M', 'W', 'D'
- 'days', 'day'
- 'hours', 'hour', 'hr', 'h'
- 'm', 'minute', 'min', 'minutes', 'T'
- 'S', 'seconds', 'sec', 'second'
- 'ms', 'milliseconds', 'millisecond', 'milli', 'millis', 'L'
- 'us', 'microseconds', 'microsecond', 'micro', 'micros', 'U'
- 'ns', 'nanoseconds', 'nano', 'nanos', 'nanosecond', 'N'



Интервалы времени

5 недель 6 дней 5 часов 37 минут 23 секунды 12 миллисекунд

```
pd.Timedelta('5W 6 days 5hr 37min 23sec 12ms')  
# Timedelta('41 days 05:37:23.012000')
```

Добавление интервала к дате

```
pd.to_datetime('2019.09.18 18:30') + pd.Timedelta('8hr 37min 23sec 12ms')  
# Timestamp('2019-09-19 03:07:23.012000')
```

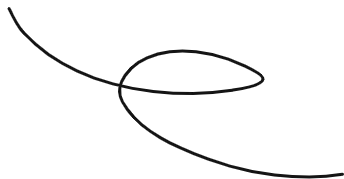
Вычитание интервала из даты

```
pd.to_datetime('2019.09.18 18:30') - pd.to_datetime('2019.09.19 18:30')  
# Timedelta('-1 days +00:00:00')
```




pd.timedelta_range —

это функция, которая позволяет сделать регулярный список дат. Она реализует функционал range для дат.



Функция `pd.timedelta_range` и ее аргументы

start

Интервал начала
отчета

end

Интервал окончания
отчета

periods

Количество
интервалов

freq

Частота отсчета

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание

Врач на протяжении дня измеряет пациенту температуру каждые 3 часа в течение 2 недель. Также пациенту необходимо спать с 11 вечера до 7 утра. Каждый день измерения температуры начинаются в 8 часов. Первое измерение 22 марта 2020 года. Определите моменты времени, когда нужно измерить пациенту температуру.

Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ



Полезные ссылки

- [pandas.DataFrame — pandas 2.2.2 documentation](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

